

MakharijPro AI

A Quran recitation companion that listens word by word: system architecture, backend design, and the machine-learning work behind the Tajweed feedback.

Project	MakharijPro AI — Real-Time Tajweed Error Detection for Quran Recitation
Developed by	Hammad Fareed & Syed Mustafa
Institution	COMSATS University Islamabad, Abbottabad Campus
Supervisor	Aisha Ajmal Khan
Document	Technical report, prepared 21 September 2026
Scope	Frontend, backend, data model, ML pipeline, evaluation, limitations

SUMMARY IN ONE PARAGRAPH

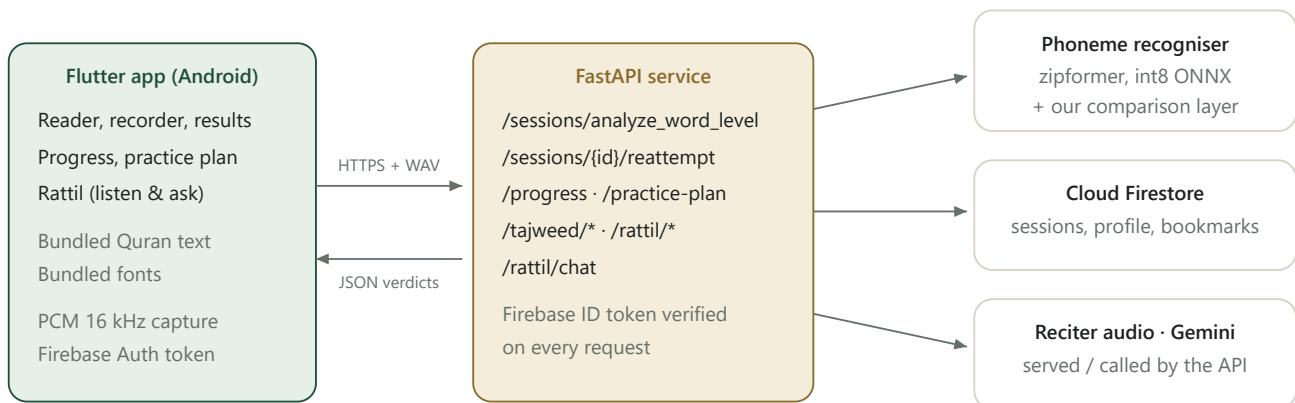
A learner opens a surah, taps the microphone and recites. The app streams the audio to our FastAPI service, which aligns the recitation against the expected phoneme sequence for exactly those ayahs, decides for each word whether it was recited and whether a Tajweed rule was broken, and returns a per-word verdict. The app shows the passage back with only the words worth reviewing marked, never a grade. Everything on screen comes from that measurement: the score is a count of words matched, progress is computed from stored sessions, and the practice plan names the actual words that were flagged.

1. What the system does

MakharijPro AI has one job: tell a learner, honestly, which words of their recitation did not sound the way that word is expected to sound, and which Tajweed rule that difference belongs to. Four capabilities support it:

- **Read.** All 114 surahs, 6,236 ayahs, bundled inside the app so the Quran renders with no network at all.
- **Recite and be checked.** Record a whole surah or a chosen range of ayahs; receive a per-word verdict.
- **Listen.** Three reciters (Abdul Rahman As-Sudais, Mishary Rashid Alafasy, Yasser Al-Dosari) with ayah, range, repeat and speed control.
- **Follow progress.** Streak, words matched over time, per-rule breakdown, and a practice plan built from the words the learner was actually flagged on.

2. Architecture



The app never talks to the recogniser, the database or Gemini directly. Every request carries a Firebase ID token, which the API verifies before touching a user's data.

2.1 Frontend (Flutter)

Concern	How it is built
Structure	Feature-first: each feature owns its screens, widgets and cubits; shared theme, widgets and services sit outside them.
State	<code>flutter_bloc</code> cubits. Recording state lives in one app-level cubit so a recitation survives navigation.
Navigation	<code>go_router</code> with a five-branch shell (Home, Quran, Rattil, Progress, Profile); each branch keeps its own stack.
Quran text	A bundled JSON asset of all 6,236 ayahs, plus bundled Amiri Quran / Amiri / Figtree fonts, so reading works offline.
Recording	<code>record</code> captures raw PCM 16-bit at 16 kHz into memory; the same samples feed the live socket and are wrapped as WAV for upload.
Networking	One API client for every service: a single resolved base address, per-request timeouts, one retry only where repeating a call is safe.

2.2 Backend (FastAPI)

Endpoint	Purpose
<code>POST /sessions/analyze_word_level</code>	The main one: audio plus surah and ayah range in, per-word verdicts out, session saved.
<code>POST /sessions/{id}/reattempt</code>	A second try at words the session already flagged; only those words may change.
<code>POST /sessions/{id}/word-feedback</code>	Records the learner disagreeing with a verdict.
<code>GET /progress, /sessions, /practice-plan, /achievements</code>	Statistics derived from the learner's own stored sessions.
<code>GET /tajweed/ayah/{s}/{a}, /tajweed/examples/{rule}</code>	Per-word Tajweed reference for highlighting and for rule examples.
<code>GET /rattil/qaris, /rattil/recitation, /rattil/word</code>	Reciter catalogue and clip URLs.
<code>POST /rattil/chat</code>	Questions the in-app parser cannot answer, passed to Gemini under tool constraints.
<code>GET /health</code>	Liveness, used by the app to find and confirm the server.

2.3 What is stored, and what is not

A session document holds the surah, the ayah range, the timestamp, the per-word verdicts, the per-rule counts and the score. **The recording itself is never uploaded to storage or kept on the server** — it exists

in memory for the length of the request and is discarded. Authentication is Firebase; the API verifies the ID token on every request and reads or writes only that user's subtree in Firestore.

3. How one recitation is judged

1. **Capture.** The app records PCM 16-bit mono at 16 kHz and, in parallel, streams it to a websocket that highlights words as they are heard. That live cursor never marks a mistake; it only shows where the reciter has reached.
2. **Upload.** On stop, the samples are wrapped as WAV and posted with the surah number and the ayah range.
3. **Recognise.** The server decodes the audio into a phoneme string with per-token timestamps.
4. **Align.** The expected phoneme sequence for exactly those ayahs is aligned against what was heard. Words beyond the point the recording stopped are returned as *not recited* — they are not mistakes.
5. **Classify.** For each recited word, the expected and heard strings are collapsed into runs of (character, repeat count) and diffed. Because the phoneme alphabet writes Tajweed features into the characters themselves, the diff recovers *which* rule differs, not merely that something did.
6. **Score and store.** The score is `words correct ÷ words recited`. Words never reached are excluded from both sides. The session is written to Firestore, and progress, practice plan and achievements are all derived from those stored sessions.

3.1 Why the rule can be named at all

The recogniser's phoneme alphabet encodes Tajweed features directly: elongation is a run of repeated carrier characters whose length is the count held, ghunnah is a run of three or more nasal characters, shaddah is a consonant written twice, ikhfa and qalqalah have dedicated symbols. Run-length diffing therefore yields a typed finding — *madd*, *ghunnah*, *shaddah*, *makhraj*, or *skipped* — with a sentence explaining it.

DESIGN RULE: FALSE POSITIVES COST MORE THAN FALSE NEGATIVES

A measured example from our own testing: the recogniser transcribes Sudais's أَلرَّحِيم with two elongation counts where the reference says four. Flagging that would tell a user a professional Qari made a mistake. So the length tolerances are deliberately loose — they fire on a dropped or grossly wrong elongation, not on the recogniser's own ± 2 jitter. An invented mistake destroys the learner's trust in every other verdict on the screen; a missed one does not.

4. The machine-learning work

The project's ML effort ran in three tracks. They are described separately because they answer different questions, and because only the first produced a model we trained ourselves.

4.1 Track A — our own Tajweed classifier (trained end to end)

Built and trained by us on Kaggle, exported as `makharijpro_tajweed_model_v1`.

Item	Detail
Training data	obadx/qdat — 1,505 clips with three binary Tajweed labels. After de-duplication by audio hash and removal of a cross-speaker duplication defect: 1,323 usable clips , split 922 / 195 / 206, grouped by speaker so no speaker appears on both sides (zero leakage, asserted in the notebook).
Features	13 MFCCs plus first-order deltas (26 dimensions), 25 ms frames, 10 ms hop, 16 kHz, silence trimmed at 25 dB; normalisation statistics fitted on the training split only.
Architecture	Shared Conv1D(32,5) → BatchNorm → Conv1D(64,5) → BatchNorm → GlobalAveragePooling, concatenated with a normalised clip-duration scalar, → Dense(64) → three sigmoid heads. 19,235 parameters.
Tasks	Separate Madd, Ghunnah (via the doubled noon), Ikhfa — each correct / incorrect.
Calibration	Per-task thresholds fitted on validation only: 0.55 / 0.40 / 0.35.

Held-out test results (206 clips):

Task	Threshold	Accuracy	F1 (macro)	Note
Ghunnah	0.40	95.6%	0.94	Strongest task; calibration transferred cleanly.
Ikhfa	0.35	79.1%	0.78	Calibration target of 10% false positives came in at 12.4% on test.
Separate Madd	0.55	74.8%	0.74	Weakest; cause identified (below), improved by adding duration.

Error analysis that changed the model. Separate Madd underperformed. We tested the obvious hypothesis — label noise among duplicate rows — and *refuted* it: the conflicted rows were already excluded in the manifest stage. The real cause was measurable: clips labelled incorrect average 7.11 s against 10.80 s for correct ones, a duration-of-elongation signal that global average pooling throws away. Adding duration as an auxiliary input lifted the task to 73.8% accuracy / 0.821 AUC, above the previous four-run band of 70.4–73.3%. A missing seed was also found and fixed, so every run after that is reproducible.

4.2 Track B — the production word-level analyser

Track A's classifier answers "was this fixed phrase recited correctly, for three rules". The app needs something different: a verdict for *every word* of *any* ayah the learner chooses. No dataset we could obtain has per-word Tajweed labels at that scale, so training that from scratch was not available to us within the project.

The shipped analyser therefore combines a third-party recogniser with a comparison layer we wrote:

- **Third-party component:** the Quran-Lab `zipformer_p_arabic_v3.1` phoneme recognition model (int8 ONNX, run through sherpa-onnx) together with its published expected-phoneme sequences for all

6,236 ayahs. Its documented phoneme error rate is about 3.65% on real audio. *We did not train this model; we integrated, measured and constrained it.*

- **Our work on top:** the word-to-phoneme mapping, the alignment of a continuous recitation against a chosen ayah range, the run-length diff that types a finding as madd / ghunnah / shaddah / makhraj / skipped, the tolerance design, the "not recited" handling, the scoring rule, the re-attempt rule, the storage schema, the practice-plan algorithm, and every guard that keeps the app from inventing a verdict.

We verified the integration end to end against eight real Rattil clips before wiring it into the app: six exact matches and 4.1% aggregate character error, consistent with the model's published figure.

4.3 Track C — evaluation on real learners

Every number the detector had been judged by came from professional Qaris, which answers the wrong question. Learners do not sound like Sudais. We evaluated against `RetaSy/quranic_audio_dataset` (6,828 crowd-sourced clips from 1,289 reciters in 82 countries).

Resolution was the first obstacle: the dataset names clips by surah name and ayah *text*, in a different orthography from our Quran asset. Exact matching resolved 18.8%; normalising three specific orthographic differences took it to 73.1%, giving **773 usable labelled clips** (353 correct, 420 incorrect).

Measured on 773 labelled learner clips	Professional Qaris	Real learners
Words wrongly flagged	1.0%	19.3%
Clips wrongly flagged	—	42.8%
Mistakes detected	—	73.8%
Balanced accuracy	—	65.5%

The gap between the two columns is the finding. It was invisible until real learner voices were measured, and two thirds of the wrongly flagged words fall into *makhraj*, the catch-all bucket that also absorbs the recogniser's own error. Detection is even across genders (male 78%, female 71%), so this is a weak detector for everyone rather than a bias against a group.

We then fitted the decision tolerances on this data — 54 settings, split by speaker, with the selection rule fixed before looking at results. **No setting beat the shipped one on both unseen splits**, and the trade-off is close to one-for-one: tightening the makhraj rule cut false positives from 46.6% to 28.1% but detection from 74.6% to 52.0%. The thresholds stayed where they were, and we documented that threshold placement is not what limits this system.

5. Rattil AI

Rattil answers requests such as "Al-Kahf by Alafasy, slowly" or "what is ghunnah?". A rule-based parser in the app handles surahs, ayahs, reciters, player commands and rules — instantly, offline, and covered by tests. Only what the parser cannot read is sent to Google Gemini, and it is given tools rather than licence: **the model is never allowed to supply Quranic text**, because a language model can misquote an ayah, and in a Quran app that is the worst possible failure. Replies generated by the model are labelled as such on screen.

6. Verification

What	Result
Frontend test suite	383 tests passing (widget, parser, scoring, audio encoding, connectivity).
Backend test suite	168 tests passing, including route-contract and no-duplicate-word tests.
Static analysis	<code>flutter analyze</code> clean.
Integration check	Eight real Rattil clips decoded and compared before the analyser was wired into the app.
Learner evaluation	773 labelled clips, reported in §4.3 with the failures included.
Device	Tested on a physical Android phone (OPPO CPH2285) against the live backend.

7. Honesty rules the product enforces

These are implementation constraints, not slogans; each one exists because the opposite was possible and was removed:

- The live cursor never flags a mistake; words not reached are never scored.
- The result is a count — "26 of 29 words matched" — never a grade or a percentage dressed as mastery.
- The waveform is drawn from the real microphone level; the processing screen shows elapsed time, not invented stages.
- Progress, streak, practice plan and achievements are computed from stored sessions only. There is no placeholder data anywhere in the app.
- When the server cannot be reached, the screen says so. It never shows a stale or fabricated result.
- A learner can mark any verdict as wrong; the disagreement is stored with the session.

8. Known limitations

- **False positives on learner audio.** About two in five correct clips receive at least one flag. This is the system's main weakness and it is measured, not estimated (§4.3).
- **Track A's classifier is limited to its dataset's fixed phrase set** and to three rules; it does not generalise to arbitrary Quran text, which is why it is not the app's word-level detector.
- **Makhraj is a catch-all.** Differences that are not elongation, nasalisation or doubling fall into it, including the recogniser's own errors.
- **Per-word accuracy is not measurable** with the corpora available to us: the learner labels are per clip, so we report detection and false-alarm rates, never per-word precision.
- **The backend runs on a development machine.** Deployment to a hosted HTTPS endpoint is the next step before the app can be used away from that network.
- Internet is required for analysis; reading the Quran is not.

9. Third-party components and attribution

Component	Role	Licence / terms
Quran-Lab zipformer_p_arabic_v3.1 + ordered_quran_phonemes	Phoneme recognition and expected sequences used by the word-level analyser (not trained by us)	As published with the model
sherpa-onnx	Runtime for the recogniser	Apache 2.0
obadx/qdat	Training data for our own classifier (Track A)	As published on Hugging Face
RetaSy/quranic_audio_dataset	Learner evaluation corpus (Track C)	As published on Hugging Face
Firebase Authentication, Cloud Firestore	Sign-in and per-user data	Google terms
Google Gemini	Answers questions the in-app parser cannot read	Google API terms
Amiri, Amiri Quran, Figtree	Typefaces bundled in the app	SIL Open Font License 1.1
Photographs	Header imagery	Unsplash License, photographers credited in-app

10. What we built, in one place

For clarity about authorship, the following are our own design and implementation:

- The Flutter application in full: reader, recorder, results, progress, practice plan, Rattil, settings, and the design system behind them.
- The FastAPI service: every endpoint, the authentication layer, the Firestore schema, and the performance work behind it.
- The word-level analysis pipeline: word mapping, range alignment, run-length diffing, rule typing, tolerance design, scoring and re-attempt rules.
- The practice-plan and achievement algorithms, derived from real session history.
- Track A end to end: dataset audit, de-duplication, leakage-safe splitting, feature pipeline, model, training, evaluation, error analysis and threshold calibration.
- Track C end to end: corpus resolution, evaluation harness, speaker-split calibration study, and the honest reporting of its negative result.
- The correction loop that stores learner disagreement as future training signal.

The one component we did not build is the phoneme recogniser used in Track B, and it is named wherever it appears.